Performance targets and measured runs on packaged fixtures

Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in `../cogant/docs/architecture/README.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	$<$ 30 s	$<$ 500 MB
100K functions	$<$ 5 min	$<$ 2 GB
1M functions	$<$ 1 hr	$<$ 2 GB (streaming)

These are architecture targets, not benchmark claims from this manuscript. They assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.5.x release, Rust acceleration is partially wired — `cogant._rust` exposes a PyO3 `connected_components` FFI for graph construction behind the `COGANT_USE_RUST` feature flag — and a pure-Python fallback

Measured runs on packaged fixtures

The following tables record measurements taken by running the shipped `RoundtripOrchestrator` (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture distributed with the package. Three fixtures are the control-positive synthetic repositories under `../cogant/examples/control_positive/` (`calculator`, `event_pipeline`, `flask_mini`); the other three are real-world code under `../cogant/examples/real_world/` (`flask_app`, a six-module Flask service; `requests_lib`, a six-module reduction of the `requests` HTTP library; and `json_stdlib`, a four-module reduction of the CPython `json` package). Each run executes the full static pipeline (ingest, parse, symbols, imports, call graph, program graph, translation, state-space compilation, GNN package build, validation). Wall-clock times were measured on a single macOS workstation with the pure-Python fallback implementations — the v0.5.x PyO3 `connected_components` FFI is disabled for this canonical run (`COGANT_USE_RUST=0`) so that these numbers correspond to the Python orchestration layer with no native crates